

# AI Usage Evidence Report — xv6 MLFQ Scheduler

Based only on the recorded project conversation and available project files.

This report documents AI use without inventing missing prompts or test results. The strongest direct evidence is the recorded user request for full modified files, the AI response describing the generated xv6 files, and the later status/test record.

## 1. Assignment context

The xv6 task requires adding a compile-time selectable MLFQ scheduler while preserving RR as the default. The specification defines four queues, 1/4/8/16 tick slices, strict priority, demotion after a full slice, unchanged priority after voluntary yield, round-robin queue 3, a 48-tick priority boost and extended procdump debugging. It also requires comparison of FIFO, RR and MLFQ using turnaround, waiting and response time, plus an MLFQ timeline plot and analysis.

## 2. Direct evidence of AI use

The most explicit user instruction in the recorded xv6 conversation is: "tell me what changes to make and give me the full modified files and i will ask you to explain me later about this project". This establishes that the user's primary xv6 AI workflow was direct generation of complete implementation files rather than only conceptual discussion. The conversation then records an AI response saying that five files were produced and were compile-verified against mit-pdos/xv6-riscv in both MLFQ and default RR modes.

## 3. Files generated and implementation scope

The recorded AI response identifies `kernel/proc.h`, `kernel/proc.c`, `kernel/trap.c`, `kernel/defs.h` and `Makefile` as the five modified files. It states that `proc.h` gained `priority`, `ticks_in_slice` and `enqueue_time`; `proc.c` gained MLFQ constants, initialization, runnable enqueue tracking, strict-priority selection with FIFO tie-breaking, slice accounting/demotion, `mlfq_boost()` and procdump information; `trap.c` invokes the boost once per tick; `defs.h` declares it; and the `Makefile` moves the SCHEDULER block to a safe location.

## 4. Scheduler design reasoning

The conversation documents a deliberate design choice to avoid linked-list queue structures. Because xv6 has a fixed process table, the scheduler can scan processes for the highest-priority RUNNABLE process and use a monotonically increasing enqueue timestamp as the within-queue FIFO ordering. The conversation also explicitly distinguishes timer-driven slice exhaustion from voluntary sleep/I/O so that voluntary yield does not incorrectly demote a process.

## 5. Build-system assistance

The user initially added an `#ifndef SCHEDULER` block to the `Makefile`. AI later identified that a later plain `CFLAGS = ...` assignment could overwrite an earlier addition, and moved the scheduler-selection block to a safe position. This is a concrete example of AI assisting with integration details rather than only writing scheduler algorithms.

## 6. User contribution

The recorded workflow shows the user cloning the xv6 code into a GitHub repository, supplying the actual project/source context, requesting complete modified files, integrating the generated changes and continuing to work with the local checkout. A later source excerpt from `kernel/proc.c` shows the integrated MLFQ enqueue/QLOG logic in the actual tree. The user therefore remained responsible for repository integration and verification.

## 7. Verification actually documented

The recorded progress review states: "\$2.1 MLFQ implementation [70] — DONE". It further states that queues, time slices, strict priority, preemption, demotion, voluntary-yield handling, priority boost and procdump were compiled clean and verified live through Ctrl+P and QLOG traces. It specifically records correct 1/4/8/16 slice demotion, boosts every 48 ticks, and voluntary yield preserving priority.

## 8. Evaluation evidence and limitations

The recorded comparison contains real RR and MLFQ values for seven processes: RR average turnaround 102.43, waiting 69.00 and response 0.00; MLFQ average turnaround 101.86, waiting 68.71 and response 0.00. FIFO is explicitly marked as not yet built. The status also says the implementation summary was not yet written, the MLFQ analysis was mostly done, and the final three-scheduler comparison was blocked on FIFO data. This report therefore does not claim that the entire xv6 evaluation was complete.

## 9. AI strengths and limitations

AI was especially useful for producing coordinated multi-file kernel changes and reasoning about scheduler state, timer behavior, queue ordering and build configuration. The limitation is that generated implementation still requires actual compilation and behavioral verification. Scheduler correctness cannot be established from source generation alone; the documented Ctrl+P/QLOG verification is therefore important evidence.

## 10. Suggested submission screenshots

Recommended evidence screenshots are: the assignment MLFQ requirements; the user's exact request for full modified files; the AI response listing the five modified files and their changes; the integrated `kernel/proc.c` excerpt showing enqueue/QLOG logic; the MLFQ DONE status showing 1/4/8/16 demotion, 48-tick boosts and voluntary-yield preservation; the RR/MLFQ comparison numbers; and the FIFO-pending status. These screenshots directly connect AI generation, implementation and verification.

## 11. Evidence Index

| Evidence                                     | Purpose                                            |
|----------------------------------------------|----------------------------------------------------|
| User request for full modified files         | Direct evidence of requested AI code generation.   |
| AI response listing five xv6 files           | Direct evidence of generated implementation scope. |
| proc.h/proc.c/trap.c/defs.h/Makefile changes | Shows concrete scheduler implementation work.      |
| Ctrl+P/QLOG verification status              | Shows recorded runtime verification.               |
| RR and MLFQ metrics                          | Shows actual evaluation data.                      |
| FIFO marked pending                          | Prevents overstating project completion.           |

Chat link: insert the actual ChatGPT conversation URL in the submission. No URL is invented here.

## 12. AI Usage Declaration

I declare that AI was used extensively during the xv6 scheduler project. My documented primary workflow was to provide the assignment and actual xv6 source context to ChatGPT and request complete modified files. AI generated substantial implementation work for the MLFQ scheduler and related build/debugging infrastructure.

I distinguish AI-generated work from my own contribution. AI generated the documented implementation files and provided design/debugging guidance. I supplied project context, integrated the generated changes, worked with the repository and participated in compilation and runtime verification. The recorded project status confirms live verification through Ctrl+P and QLOG traces.

I have not treated AI output as automatically correct, and I do not claim that the FIFO comparison or final three-scheduler report was complete because the recorded status explicitly identifies FIFO as pending. Where exact earlier prompts are unavailable, I have not reconstructed them.